

Objetivos da aula:

- Compreender o papel da extração de características na transformação de texto em representações numéricas.
- Representar textos utilizando modelos clássicos baseados em contagem, como Bag-of-Words, frequência de termos e n-grams.
- Explicar o funcionamento e a motivação do modelo TF-IDF como técnica de ponderação de termos.
- Identificar as limitações dos modelos baseados em contagem e ponderação estatística.
- Compreender o conceito de Word Embeddings como representações distribuídas no espaço vetorial.
- Utilizar embeddings pré-treinados para analisar similaridade, vizinhança semântica e analogias entre palavras.
- Avaliar criticamente limitações de Word Embeddings, como polissemia, viés, dependência do corpus e cobertura lexical.

Índice:

- i. Representação Vetorial do Texto
- ii. Ponderação de Termos e o Modelo TF-IDF
- iii. Word Embeddings e Representações Distribuídas

i. Representação Vetorial do Texto

Introdução **slide 6**

Após as etapas de pré-processamento textual apresentadas nas aulas anteriores, surge uma questão central no PLN:

Como transformar texto em uma forma numérica adequada para algoritmos de Aprendizado de Máquina?

A resposta envolve o processo de **extração de características**, no qual textos — naturalmente simbólicos — são convertidos em **representações vetoriais**. Essas representações permitem que documentos e palavras sejam tratados como objetos matemáticos, viabilizando tarefas como classificação, agrupamento e recuperação de informação.

Este capítulo apresenta as abordagens **clássicas de representação textual baseadas em contagem**, que constituem o ponto de partida conceitual para técnicas mais avançadas discutidas posteriormente.

Texto como dado estruturado **slide 7**

Embora o texto seja produzido para consumo humano, algoritmos computacionais exigem **estruturas numéricas bem definidas**. Assim, no PLN, o texto processado passa a ser tratado como um **dado estruturado**, tipicamente na forma de:

- listas de tokens;

- vetores numéricos;
- matrizes documento-termo.

Nesse contexto, cada documento é representado por um vetor de dimensão fixa, em que cada dimensão corresponde a uma **característica extraída do texto**, como a ocorrência de uma palavra ou sequência de palavras. Essa transformação é essencial, pois algoritmos de Aprendizado de Máquina não operam diretamente sobre texto, mas sobre **números e estruturas vetoriais**.

1 Exemplo de Lista de Tokens

Texto original:

"Eu gosto de aprender PLN"

◆ Após tokenização:

```
["eu", "gosto", "de", "aprender", "pln"]
```

◆ Após remover stopwords:

```
["gosto", "aprender", "pln"]
```

👉 Isso é uma **lista de tokens**:

Uma lista Python contendo as palavras do texto.

2 Exemplo de Vetor Numérico (Bag-of-Words)

Suponha 3 documentos:

```
docs = [  
    "eu gosto de pln",  
    "eu gosto de python",  
    "pln é interessante"  
]
```

◆ Vocabulário criado:

```
["eu", "gosto", "pln", "python", "interessante"]
```

◆ Vetores numéricos

Documento 1 → "eu gosto de pln"

```
[1, 1, 1, 0, 0]
```

Documento 2 → "eu gosto de python"

```
[1, 1, 0, 1, 0]
```

Documento 3 → "pln é interessante"

```
[0, 0, 1, 0, 1]
```

👉 Cada posição representa uma palavra do vocabulário

👉 Cada número representa a frequência

Isso é um **vetor numérico**.

3 Matriz Documento-Termo

Agora juntando tudo:

Documento	eu	gosto	pln	python	interessante
Doc 1	1	1	1	0	0
Doc 2	1	1	0	1	0
Doc 3	0	0	1	0	1

👉 Linhas = documentos

👉 Colunas = termos

👉 Valores = frequência

Isso é a **Matriz Documento-Termo (DTM)**.

4 Exemplo com TF-IDF

Agora os valores deixam de ser contagem simples:

Documento	eu	gosto	pln	python	interessante
Doc 1	0.2	0.3	0.6	0	0
Doc 2	0.2	0.3	0	0.7	0
Doc 3	0	0	0.5	0	0.8

👉 Aqui os números representam **importância**, não contagem.

5 Exemplo com Word Embeddings (fastText)

Com **fastText**, cada palavra vira um vetor denso:

"pln" → [0.12, -0.45, 0.33, 0.88, ...]

Vetor com 50, 100 ou 300 dimensões.

Diferente do BoW:

- BoW → muitos zeros
- Embeddings → números reais densos
- Capturam semântica

Resumo Didático para Aula

Representação	Exemplo
Lista de tokens	["gosto", "aprender", "pln"]
Vetor numérico	[1, 1, 0, 0, 1]
Matriz documento-termo	Tabela documentos × palavras
Embedding	[0.23, -0.55, 0.98, ...]

Modelo Bag-of-Words (BoW) slide 8

O modelo Bag-of-Words (BoW) é uma das abordagens mais simples e fundamentais para representação textual. A ideia central do BoW é representar um texto como um **conjunto de palavras**, ignorando completamente a ordem em que elas aparecem. O modelo considera apenas:

- quais palavras aparecem no texto;
- quantas vezes elas aparecem (em algumas variantes).

Exemplo simples: slide 9

Considere o vocabulário:

```
["produto", "bom", "ruim", "qualidade"]
```

E o texto processado:

```
["produto", "bom", "qualidade"]
```

A representação BoW seria:

```
[1, 1, 0, 1]
```

Cada posição do vetor corresponde a uma palavra do vocabulário, e o valor indica sua presença no documento.

Apesar de sua simplicidade, o BoW é conceitualmente importante, pois estabelece a ponte entre **texto e vetores numéricos**.

De onde vieram os números?

Eles vêm da **contagem de frequência das palavras** em cada documento.

Isso é o modelo **Bag-of-Words (BoW)**.

Construindo o Vetor Manualmente

Documento 1

Texto: "eu gosto de pln"

Contamos quantas vezes cada palavra do vocabulário aparece:

Palavra	Aparece?	Quantidade
eu	sim	1
gosto	sim	1
pln	sim	1
python	não	0
interessante	não	0

Resultado:

```
[1, 1, 1, 0, 0]
```

Frequência de termos **slide 10**

Uma extensão natural do BoW consiste em utilizar **frequências de termos**, em vez de apenas indicar presença ou ausência.

Nesse modelo, cada dimensão do vetor indica **quantas vezes** um termo aparece no documento.

Exemplo:

Texto processado:

```
["produto", "produto", "bom"]
```

Vocabulário:

```
["produto", "bom"]
```

Vetor de frequências:

```
[2, 1]
```

Essa abordagem permite capturar a importância relativa de termos dentro de um documento, mas introduz novos desafios, como a influência do tamanho do texto e a dominância de palavras muito frequentes.

N-grams **slide 11**

Até este ponto, consideramos apenas palavras isoladas (*unigrams*). No entanto, muitas informações relevantes da linguagem estão associadas a **sequências de palavras**.

Os n-grams são sequências contíguas de n tokens extraídos de um texto.

Exemplo:

Texto:

```
["não", "gostei", "produto"]
```

```
Unigrams (n = 1): ["nao", "gostei", "produto"]
```

```
Bigrams (n = 2): ["nao gostei", "gostei produto"]
```

O uso de n-grams permite capturar informações locais de contexto, como expressões fixas e negações simples. Entretanto, o aumento do valor de n faz com que o vocabulário cresça rapidamente, impactando o custo computacional.

Em termos simples:

É um conjunto de itens consecutivos, sem pular nenhum no meio.

Exemplo simples (lista de números)

Lista:

```
[10, 20, 30, 40, 50]
```

Sequências contíguas possíveis:

- [10, 20]
- [20, 30, 40]
- [30, 40, 50]

✗ Não é contígua:

- [10, 30] → pulou o 20
- [20, 40] → pulou o 30

Quando dizemos que aumentar o **n** nos **n-grams** aumenta o custo computacional, estamos falando principalmente de **três tipos de custo**:

1 Custo de Memória (RAM)

Quanto maior o **n**, maior o número de combinações possíveis de palavras.

Exemplo simples:

Frase:

"eu gosto de aprender pln"

Unigramas (n=1): → 5 termos

eu, gosto, de, aprender, pln

Bigramas (n=2): → 4 termos

eu gosto
gosto de
de aprender
aprender pln

Trigramas (n=3): → 3 termos

eu gosto de
gosto de aprender
de aprender pln

Isso parece pequeno...

Mas em um corpus real com **50.000 palavras únicas**, o crescimento explode.

Se o vocabulário de unigramas for:

$V=50.000$

O número teórico de bigramas possíveis pode chegar a:

$V^2=2.500.000.000$

Ou seja: **bilhões de combinações possíveis**.

Mesmo que nem todas apareçam, o vocabulário cresce muito.

👉 Resultado:

Mais memória para armazenar matriz documento-termo.

2 Custo de Processamento (CPU)

Ao usar n-grams:

- O algoritmo precisa gerar todas as combinações contíguas.
- A matriz fica muito maior.
- O tempo de treino do modelo aumenta.

Exemplo:

BoW com unigramas → matriz 10.000 x 5.000

BoW com trigramas → pode virar 10.000 x 200.000

Mais colunas = mais operações matemáticas.

3 Custo de Armazenamento

Se você salvar a matriz em disco:

- Arquivo fica muito maior.
- Modelos treinados ocupam mais espaço.

Vídeo sobre o assunto: <https://www.youtube.com/watch?v=GiyMGBuu45w>

⚠ Limitações dos modelos baseados em contagem slide 12

Embora modelos baseados em contagem sejam simples e eficazes em diversas aplicações, eles apresentam limitações estruturais importantes:

- Ausência de semântica: palavras semanticamente semelhantes são tratadas como completamente distintas.
- Perda de ordem global: a estrutura sintática do texto é ignorada.
- Alta dimensionalidade: o número de características cresce com o vocabulário.
- Sensibilidade lexical: variações morfológicas e sinônimos não são capturados naturalmente.

Essas limitações motivaram o desenvolvimento de técnicas que buscam atribuir pesos diferenciados aos termos e, posteriormente, capturar relações semânticas mais profundas.

ii. Ponderação de Termos e o Modelo TF-IDF slide 13

Introdução

No capítulo anterior, foram apresentadas representações textuais baseadas em **contagem de ocorrências**, como Bag-of-Words e n-grams. Embora essas abordagens permitam transformar texto em vetores numéricos, elas tratam todos os termos como igualmente importantes, o que raramente é desejável.

Na prática, algumas palavras carregam mais informação do que outras. Palavras muito frequentes na língua tendem a aparecer em praticamente todos os documentos, contribuindo pouco para diferenciá-los.

Este capítulo apresenta técnicas de **ponderação de termos**, com destaque para o modelo **TF-IDF**, que busca equilibrar a importância local de um termo em um documento com sua relevância global em um conjunto de textos.

A necessidade de ponderar termos

Considere um conjunto de documentos que tratam de um mesmo domínio. Termos como artigos, preposições ou palavras muito genéricas tendem a aparecer com alta frequência em todos os textos.

Embora essas palavras sejam frequentes, elas possuem **baixo poder discriminativo**. Por outro lado, termos que aparecem em poucos documentos tendem a ser mais informativos, pois ajudam a caracterizar temas ou conteúdos específicos.

A ponderação de termos surge para lidar com esse problema, atribuindo:

- **pesos menores** a termos muito comuns;
- **pesos maiores** a termos mais raros e informativos.

Term Frequency (TF) **slide 14**

A frequência de termos (*Term Frequency – TF*) mede o quanto um termo aparece em um documento específico.

De forma conceitual, o TF reflete a **importância local** de um termo dentro de um texto. Quanto mais vezes um termo aparece em um documento, maior tende a ser sua contribuição para a representação desse documento.

Entretanto, o TF isoladamente não distingue entre termos comuns a todos os documentos e termos realmente característicos de um texto específico.

Exemplo de Term Frequency (TF)

Corpus com 2 documentos

Documento 1:

`"pln é interessante e pln é útil"`

Documento 2:

`"machine learning é interessante"`

Calculando o TF no Documento 1

Primeiro, contamos as palavras do Documento 1.

Tokens:

`pln, é, interessante, e, pln, é, útil`

Total de palavras = 7

Agora contamos a frequência do termo "pln":

- "pln" aparece **2 vezes**

Fórmula do TF

$$TF(t, d) = \frac{\text{número de vezes que o termo aparece no documento}}{\text{total de termos no documento}}$$

◆ TF de "pln" no Documento 1:

$$TF(pln, Doc1) = \frac{2}{7} \approx 0,28$$

◆ TF de "é" no Documento 1:

"é" aparece 2 vezes:

$$TF(é, Doc1) = \frac{2}{7} \approx 0,28$$

Inverse Document Frequency (IDF) **slide 15**

A frequência inversa de documentos (*Inverse Document Frequency – IDF*) mede o quão raro ou comum um termo é em um conjunto de documentos.

A intuição por trás do IDF é simples:

- termos que aparecem em muitos documentos são pouco informativos;
- termos que aparecem em poucos documentos são mais relevantes.

Assim, o IDF atua como um fator de penalização para palavras muito frequentes no corpus, reduzindo sua influência na representação vetorial.

Exemplo Prático

◆ Corpus com 3 documentos

Documento 1:

"pln é interessante"

Documento 2:

"machine learning é interessante"

Documento 3:

"pln é útil"

Total de documentos: N=3



Fórmula do IDF

Forma mais usada:

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

Onde:

- N = número total de documentos
- $df(t)$ = número de documentos que contêm o termo

1 Calculando para o termo "pln"

"pln" aparece em:

- Documento 1
- Documento 3

Então:

$$df(pln) = 2$$

$$IDF(pln) = \log \left(\frac{3}{2} \right)$$

$$IDF(pln) \approx \log(1,5)$$

$$IDF(pln) \approx 0,18$$

2 Calculando para o termo "é"

"é" aparece em:

- Documento 1
- Documento 2
- Documento 3

$$df(é) = 3$$

$$IDF(é) = \log \left(\frac{3}{3} \right)$$

$$IDF(é) = \log(1)$$

$$IDF(é) = 0$$

👉 Ou seja: não tem poder de diferenciação.

3 Calculando para o termo "machine"

"machine" aparece apenas no Documento 2.

$$df(machine) = 1$$

$$IDF(machine) = \log\left(\frac{3}{1}\right)$$

$$IDF(machine) = \log(3)$$

$$IDF(machine) \approx 0,47$$

👉 Termo raro → IDF maior.

O modelo TF-IDF slide 16

O modelo TF-IDF combina as duas medidas anteriores, multiplicando a importância local de um termo (TF) pela sua importância global (IDF).

De forma intuitiva:

Um termo será considerado importante se aparecer com frequência em um documento, mas em poucos documentos do corpus.

O resultado é uma **matriz documento-termo ponderada**, na qual cada valor indica a relevância de um termo em um documento específico.

Essa ponderação melhora significativamente a capacidade discriminativa das representações textuais em comparação com modelos puramente baseados em contagem.

Interpretação conceitual

No TF-IDF:

- palavras muito comuns tendem a ter pesos próximos de zero;
- termos específicos de um documento recebem pesos mais elevados;
- documentos passam a ser representados por vetores que enfatizam seu conteúdo distintivo.

Por esse motivo, o TF-IDF é amplamente utilizado em tarefas como:

- classificação de documentos;
- recuperação de informação;
- análise exploratória de coleções textuais.

Limitações do TF-IDF

Apesar de representar um avanço em relação aos modelos baseados em contagem, o TF-IDF ainda apresenta limitações importantes:

- Ausência de semântica: palavras semanticamente semelhantes continuam sendo tratadas como distintas;

- Representação esparsa: o espaço vetorial permanece de alta dimensionalidade;
- Dependência do corpus: os pesos de IDF variam conforme o conjunto de documentos utilizado;
- Ignora contexto: a ordem das palavras não é considerada.

Essas limitações indicam que, embora o TF-IDF seja eficaz, ele não captura relações de significado mais profundas entre palavras.

iii. Word Embeddings e Representações Distribuídas **slide 17**



Introdução

Nos capítulos anteriores, foram apresentadas técnicas de extração de características baseadas em **contagem** (Bag-of-Words, frequência de termos e n-grams) e em **ponderação estatística** (TF-IDF). Essas abordagens permitem transformar texto em vetores numéricos, mas apresentam uma limitação estrutural importante:

Palavras são tratadas como símbolos independentes, sem relação semântica explícita entre si.

Como consequência, palavras semanticamente próximas, como “bom” e “ótimo”, são representadas como vetores completamente distintos, enquanto palavras semanticamente opostas não são naturalmente diferenciadas de termos apenas não relacionados.

Os Word Embeddings surgem como uma alternativa mais expressiva, introduzindo representações capazes de capturar similaridade semântica de forma explícita.

Este capítulo apresenta os princípios conceituais dos Word Embeddings, enquanto sua aplicação prática é explorada no notebook `Aula 3 - Exemplo fastText PT.ipynb`, que utiliza embeddings *fastText* em português.



Ideia central dos Word Embeddings

Word Embeddings são **representações vetoriais densas e contínuas de palavras**, aprendidas automaticamente a partir de grandes corpora textuais.

Diferentemente de modelos baseados em contagem, nos quais cada palavra ocupa uma dimensão independente, os embeddings projetam todas as palavras em um **mesmo espaço vetorial de baixa dimensão**, no qual:

- cada palavra é representada por um vetor numérico;
- palavras semanticamente semelhantes tendem a ocupar posições próximas;
- relações linguísticas emergem da geometria do espaço vetorial.

Essa abordagem está fundamentada na chamada **hipótese distribucional**, segundo a qual:

Palavras que ocorrem em contextos semelhantes tendem a ter significados semelhantes.

No notebook `Aula 3 - Exemplo fastText PT.ipynb`, essa ideia é observada empiricamente por meio da análise de similaridade e vizinhança semântica entre palavras.

Espaço vetorial semântico **slide 18**

Do ponto de vista matemático, um modelo de Word Embeddings define um espaço vetorial $\mathbb{R}^n$, no qual cada palavra é representada por um vetor de dimensão fixa ($n = 300$, neste caso).

Nesse espaço:

- o significado não está associado a dimensões isoladas;
- o significado emerge da **posição relativa** entre vetores;
- a similaridade semântica é interpretada geometricamente.

Assim, comparar palavras deixa de ser uma operação simbólica e passa a ser uma operação geométrica, baseada na proximidade entre vetores no espaço.

Similaridade e vizinhança semântica **slide 19**

Uma consequência direta da representação vetorial é a possibilidade de medir **similaridade semântica** entre palavras e identificar **vizinhos semânticos**.

De forma geral:

- palavras semanticamente próximas apresentam vetores mais alinhados;
- palavras semanticamente distantes apresentam vetores com direções diferentes.

No notebook, observamos esses fenômenos ao calcular similaridades entre pares de palavras e ao recuperar termos semanticamente próximos de uma palavra-alvo.

Esses experimentos tornam concreta a ideia de que **significado emerge da geometria do espaço vetorial**, e não de regras linguísticas explícitas.

Analogias vetoriais

Além da similaridade, Word Embeddings podem capturar **regularidades linguísticas globais**, que se manifestam na forma de analogias do tipo:

$$a : b :: c : ?$$

A intuição é que certas relações linguísticas correspondem a **deslocamentos vetoriais aproximadamente constantes** no espaço semântico.

No notebook, essa propriedade é explorada por meio de operações vetoriais simples, permitindo analisar por que algumas analogias funcionam bem enquanto outras falham, especialmente em função do corpus de treinamento e do vocabulário disponível.

Essa etapa reforça que os embeddings capturam **regularidades estatísticas da língua**, e não regras gramaticais formais.

Embeddings pré-treinados e o fastText **slide 20**

Nesta aula, utilizamos embeddings pré-treinados, em vez de treinar modelos do zero.

Os vetores empregados no notebook foram gerados com o modelo **fastText**, a partir de textos em língua portuguesa coletados pelo projeto **Common Crawl**. O treinamento foi realizado sobre um corpus contendo **bilhões de ocorrências de palavras (tokens)**, resultando em um vocabulário final de aproximadamente **2 milhões de palavras distintas (types)**, cada uma associada a um vetor de 300 dimensões.

O uso de embeddings pré-treinados permite explorar propriedades semânticas ricas sem o custo computacional associado ao treinamento de modelos em larga escala.

O que é o Common Crawl?

É um projeto que mantém um repositório público de grandes volumes de textos coletados da web, incluindo:

- páginas de notícias,
- blogs,
- fóruns,
- textos institucionais,
- conteúdos educacionais,
- outros textos públicos disponíveis na internet.

Esses dados são coletados periodicamente por meio de **web crawling** e disponibilizados para fins de pesquisa.

No caso do fastText, o treinamento foi realizado sobre um corpus contendo bilhões de ocorrências de palavras (tokens) em língua portuguesa.

Corpus × Vocabulário (distinção fundamental)

É importante distinguir dois conceitos diferentes:

- **Corpus**: total de palavras observadas durante o treinamento (tokens). No caso do Common Crawl, esse número está na ordem de **bilhões**;
- **Vocabulário**: conjunto de palavras distintas que receberam vetores explícitos no modelo final (types). No arquivo `cc.pt.300.vec.gz`, esse vocabulário contém aproximadamente **2 milhões de palavras distintas**, cada uma associada a um vetor de 300 dimensões.

O modelo foi treinado sobre bilhões de ocorrências, mas o resultado final materializa vetores apenas para as palavras distintas mais relevantes (~2 milhões).

O uso de um corpus amplo e heterogêneo como o Common Crawl implica que:

- os embeddings capturam **regularidades semânticas gerais da língua**;
- palavras frequentes possuem representações mais estáveis;
- termos raros ou muito específicos podem ter representações menos precisas;
- os vetores podem refletir **viéses presentes na web**, como estereótipos culturais ou sociais.

Essas características reforçam que embeddings pré-treinados devem ser compreendidos como modelos estatísticos da linguagem, e não como representações neutras ou completas do significado.

 Dimensionalidade dos embeddings slide 22

No modelo utilizado, cada palavra é representada por um vetor de 300 dimensões.

A dimensionalidade de um embedding é um hiperparâmetro, escolhido de modo a equilibrar:

- capacidade de representação semântica;
- estabilidade estatística;
- custo computacional e de memória.

Dimensões maiores permitem capturar relações mais complexas, mas exigem mais recursos. O valor 300 consolidou-se como um padrão na literatura por oferecer um bom compromisso entre esses fatores.

É importante ressaltar que as dimensões não possuem interpretação linguística direta: o significado emerge apenas da combinação das dimensões e da posição relativa dos vetores no espaço.

O que é uma dimensão em um embedding?

Em termos matemáticos, um embedding de dimensão 300 representa cada palavra como um vetor em um espaço vetorial $\mathbb{R}^{300}$, isto é:

$$\vec{w} = (w_1, w_2, w_3, \dots, w_{300})$$

Cada uma dessas 300 dimensões corresponde a um **valor numérico real**, aprendido automaticamente durante o treinamento do modelo.

É importante destacar que:

- as dimensões não possuem significado linguístico explícito;
- não existe uma dimensão “do gênero”, “do sentimento” ou “do tópico”;
- o significado emerge da combinação das dimensões e da posição relativa dos vetores no espaço.

Exemplo ilustrativo em baixa dimensão (3D) slide 23

Para fins didáticos, considere um exemplo simplificado em apenas 3 dimensões, que podemos visualizar intuitivamente.

Suponha os seguintes vetores (fictícios):

- bom $\rightarrow (0.8, 0.1, 0.2)$
- ótimo $\rightarrow (0.75, 0.15, 0.25)$
- ruim $\rightarrow (-0.7, -0.2, 0.1)$

Nesse espaço tridimensional:

- os vetores de “**bom**” e “**ótimo**” apontam para direções semelhantes;
- o vetor de “**ruim**” aponta para uma direção diferente;
- a proximidade angular entre “bom” e “ótimo” indica **similaridade semântica**;
- a distância angular entre “bom” e “ruim” indica **oposição semântica**.

Embora esse exemplo use apenas 3 dimensões, a lógica é exatamente a mesma em 300 dimensões — apenas em um espaço que não conseguimos visualizar diretamente.

 Limitações dos Word Embeddings slide 24

Apesar de representarem um avanço significativo em relação a modelos baseados em contagem, os Word Embeddings apresentam limitações importantes:

- **Representação estática:** cada palavra possui um único vetor, independentemente do contexto;
- **Polissemia:** palavras com múltiplos sentidos compartilham a mesma representação. Ex.: banco (instituição × assento) e manga (fruta × roupa);
- **Dependência do corpus:** o significado capturado reflete os dados de treinamento;
- **Viés:** estereótipos presentes no corpus podem ser incorporados aos vetores.

Essas limitações motivaram o desenvolvimento de modelos contextuais mais recentes, que serão discutidos em aulas posteriores.

 Exercícios**Exercício 1** - Comparação entre Bag-of-Words, TF-IDF e Word Embeddings

Considere os dois textos a seguir, após pré-processamento (remoção de stopwords, normalização e tokenização):

Texto A: ["produto", "bom", "ser", "produto", "qualidade"]

Texto B: ["produto", "ruim", "defeito"]

Considere o vocabulário:

["produto", "bom", "ruim", "ser", "qualidade", "defeito"]

Responda às questões a seguir.

Questão 1 – Bag-of-Words

- Represente os dois textos usando o modelo Bag-of-Words.
- Com base apenas nesses vetores, é possível afirmar que os textos possuem significados semelhantes ou diferentes? Justifique.

Resposta:

- Representação Bag-of-Words: Texto A [x, x, x, x, x, x] e Texto B [x, x, x, x, x, x]
- xxxxxxxxxxxxxxxxxxxxxxxx

Questão 2 – Frequência de termos

Represente o texto usando frequência de termos.

Resposta:

Vetor de frequência: Texto A [x, x, x, x, x, x] e Texto B [x, x, x, x, x, x]

Questão 3 – TF-IDF (conceitual)

Suponha que, em um corpus maior, a palavra "produto" apareça em praticamente todos os documentos, enquanto "defeito" apareça em poucos.

- Qual termo tende a receber maior peso no TF-IDF?
- Justifique o motivo, sem usar fórmulas.

Resposta:

- xx.
- xx

Questão 4 – Word Embeddings (conceitual)

Considere agora um modelo de Word Embeddings bem treinado.

- a) O que se espera da distância vetorial entre:
- “bom” e “ótimo”
 - “bom” e “ruim”
- b) Por que esse tipo de relação não pode ser capturada adequadamente por BoW ou TF-IDF?

Resposta:

- a) “bom” e “ótimo” → xxxxxxxxxxxxxxxx
 “bom” e “ruim” → xxxxxxxxxxxxxxxxxxxxxx
- b) xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

Exercício 2 – Do texto bruto ao vetor (exercício-ponte)

Objetivo: Evidenciar que o notebook da Aula 3 começa exatamente onde a Aula 2 termina.

Enunciado: Considere o seguinte texto bruto:

“O produto não é bom, mas a qualidade do material é excelente.”

- a) Aplique as etapas de pré-processamento vistas na Aula 2:
- normalização (caixa baixa);
 - remoção de pontuação;
 - tokenização;
 - remoção de stopwords.
- b) Explique por que apenas esses tokens, e não o texto original, são utilizados como entrada no notebook de Word Embeddings da Aula 3.

Resposta:

- a) Etapas de pré-processamento

Normalização (caixa baixa):

xx

Remoção de pontuação:

xx

Tokenização:

xx

Remoção de stopwords (admitindo uma lista típica do português):

xx

